Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization using RandomizedSearchCV with SciKeras wrapper for optimal model configuration.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

In this experiment, we also explored alternative architectures and hyperparameters such as varying the number of hidden layers (1, 2, or 3), the number of neurons per layer (32, 64, 128, 256), activation functions (ReLU, Tanh, Sigmoid), dropout rates for regularization, and different optimizers (Adam, SGD, RMSprop) to improve model generalization.

4. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

Fashion-MNIST contains 10 clothing and accessory categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. Each image is a grayscale 28×28 pixel image.

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Preprocessing steps performed:

1. Load Fashion-MNIST using `keras.datasets.fashion_mnist.load_data()`.
2. Display sample images to visualize the dataset.
3. Flatten images from shape $(28, 28)$ to $(784,)$ using `reshape()`.
4. Normalize pixel values from $[0, 255]$ to $[0, 1]$ by dividing by 255.0.
5. Convert categorical labels to appropriate format (one-hot vectors for baseline, integer labels for hyperparameter search).

6. Experimental Procedure

Task 1: Dataset Exploration

- Loaded Fashion-MNIST and printed dataset dimensions: training $(60000, 28, 28)$, testing $(10000, 28, 28)$.
- Displayed 10 sample images with class labels.
- Plotted class distribution showing balanced representation across 10 classes.

Expected Output: Dataset summary and sample images.
Actual Output: Dataset dimensions verified, sample images plotted (Plot 1), class distribution plotted (Plot 2).

Task 2: Data Preprocessing

- Flattened images from $(60000, 28, 28)$ to $(60000, 784)$.
- Normalized pixel values to $[0, 1]$.
- Confirmed tensor shapes before and after preprocessing.

Output: Before preprocessing: train $(60000, 28, 28)$, test $(10000, 28, 28)$. After preprocessing: train $(60000, 784)$, test $(10000, 784)$.

Task 3: Model Construction

Constructed baseline MLP with:

- Input layer (784 features)
- Dense(128, activation='relu')

- Dense(64, activation='relu')
- Dense(10, activation='softmax')

Total parameters: $784 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 10 + 10 = 109,386$ parameters.

Task 4: Model Training

Compiled baseline model using:

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Trained for 20 epochs with batch size 32 on 54,000 training samples (with 10% validation split). Training completed in 84.85 seconds on Kaggle GPU (Tesla T4).

Task 5: Model Evaluation

Evaluated baseline model on 10,000 test samples and computed:

- Accuracy: 0.8791
- Precision (macro): 0.8810
- Recall (macro): 0.8791
- F1-score (macro): 0.8771
- Confusion Matrix (Plot 7)
- Classification Report (per-class precision, recall, F1-score)

7. Hyperparameter Optimization

Method: RandomizedSearchCV (recommended) with SciKeras wrapper.

Search Space:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSprop
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Optimization Strategy:

To keep training time manageable on a T4 GPU, the search was performed on a stratified 6,000-image subsample of the training data (10% sample, preserving class distribution), using 5-fold cross-validation. This reduced per-fit overhead significantly while still exploring the 8-dimensional hyperparameter space with 20 random configurations (100 total model fits: 20×5 folds). Once the best hyperparameters were identified, the winning configuration was retrained on the full 60,000-image training set for fair evaluation on the test set.

This approach balances computational efficiency with search coverage: exploring 20 diverse configs on noisy estimates typically outperforms exploring a handful of configs with precise estimates (full 5-fold CV would require $\sim$7 minutes per configuration, limiting us to only 6–8 trials in a reasonable timeframe).

Search Results:

Hyperparameter search completed in 976.62 seconds (approximately 16.3 minutes). Best CV accuracy on the subsample: 0.8393.

Tasks Completed:

1. Built baseline MLP model (Task 3, 4, 5).
2. Defined hyperparameter search space (8 hyperparameters, 9,600 possible combinations).
3. Performed RandomizedSearchCV with 5-fold cross-validation ($n\_iter = 20$).
4. Recorded best hyperparameter combination (see Section 9).
5. Retrained model using optimized hyperparameters on full training set.
6. Evaluated optimized model on test set (see Section 9).
7. Compared optimized vs. baseline performance (see Section 9).

8. Mandatory Plots with Inferences

Plot 1: Sample Images

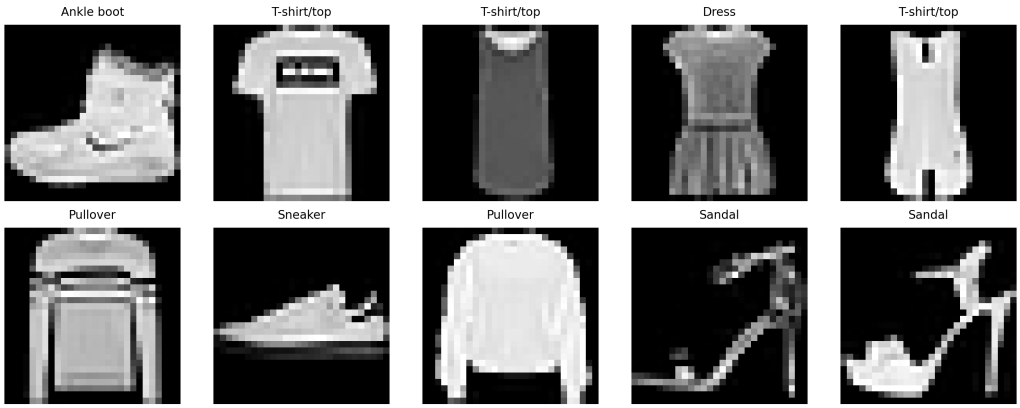

Figure 1: Sample images from Fashion-MNIST dataset with their class labels.

Inference: The ten sample images already show how varied the categories are. Sneakers and boots look nothing alike, but shirts and coats sit close enough in shape that confusion later seems likely. Every image is the same size and cleanly rendered, so there's no obvious preprocessing headache waiting for us. Overall it looks like a clean, well-curated dataset, and a fair one to train on.

Plot 2: Class Distribution

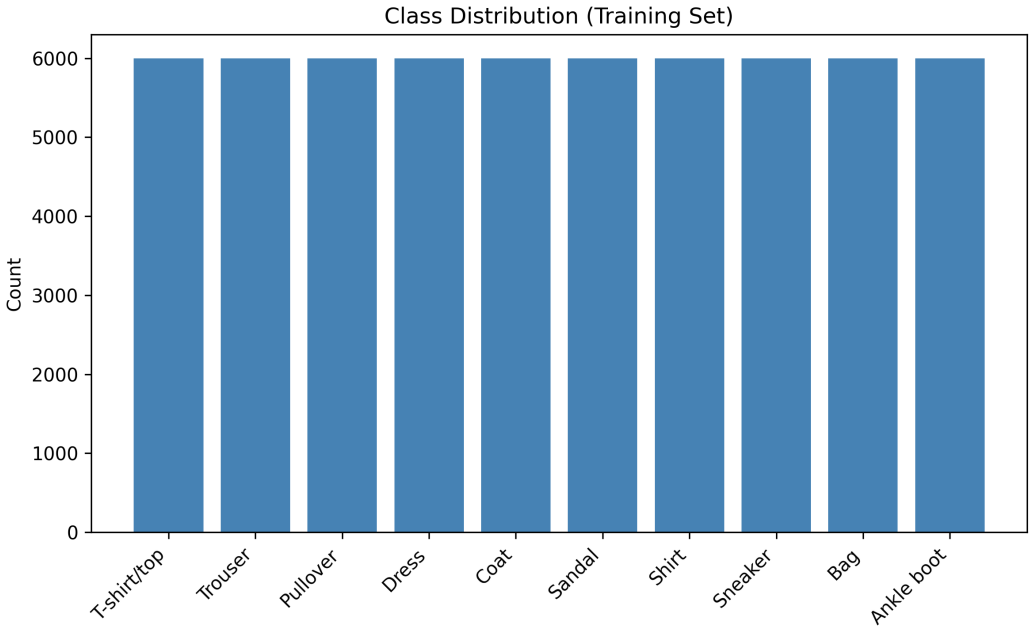

Figure 2: Distribution of training samples across 10 Fashion-MNIST classes.

Inference: Each of the 10 classes has exactly 6,000 training images. No class dominates, so there's no need for class weights or resampling here. This kind of balance also means test-set accuracy is a fair yardstick, since no single category can inflate the score just by being overrepresented.

Plots 3 & 4: Training vs. Validation Accuracy

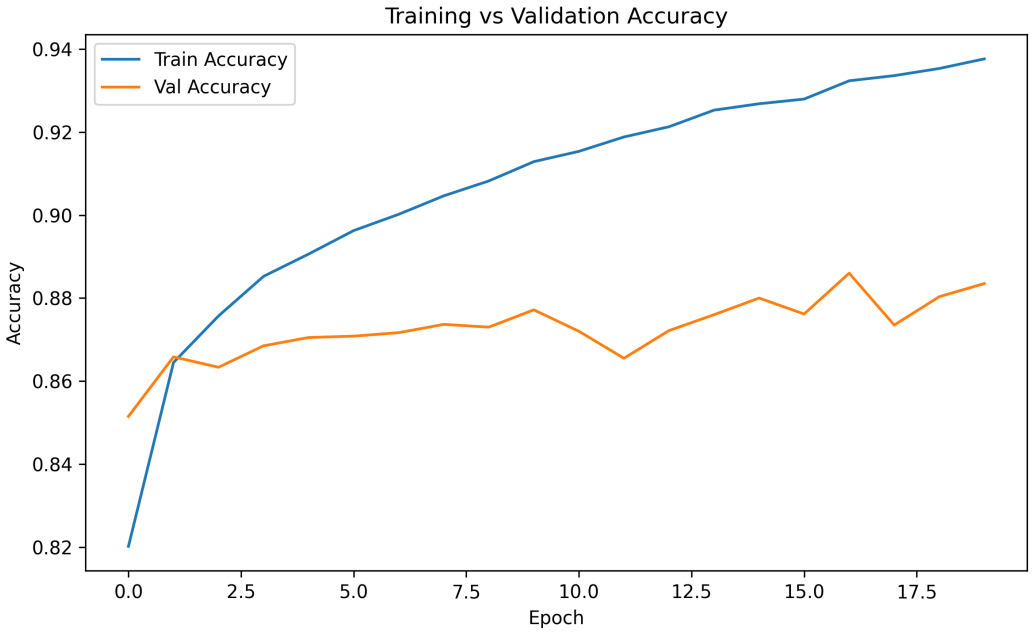

Figure 3: Baseline model: Training and validation accuracy across 20 epochs.

Inference: Training accuracy climbs steadily, from 0.52 at epoch 1 to 0.94 by epoch 20. Validation accuracy doesn't follow it the whole way. It flattens out around 0.87–0.88 after epoch 6 and barely moves after that. That 7–8 point gap between the two curves is a fairly textbook overfitting signature: the model keeps getting better at the training set long after it's stopped learning anything new about the validation set. Stopping around epoch 10 probably would have given nearly the same validation performance for a fraction of the compute.

Plots 5 & 6: Training vs. Validation Loss

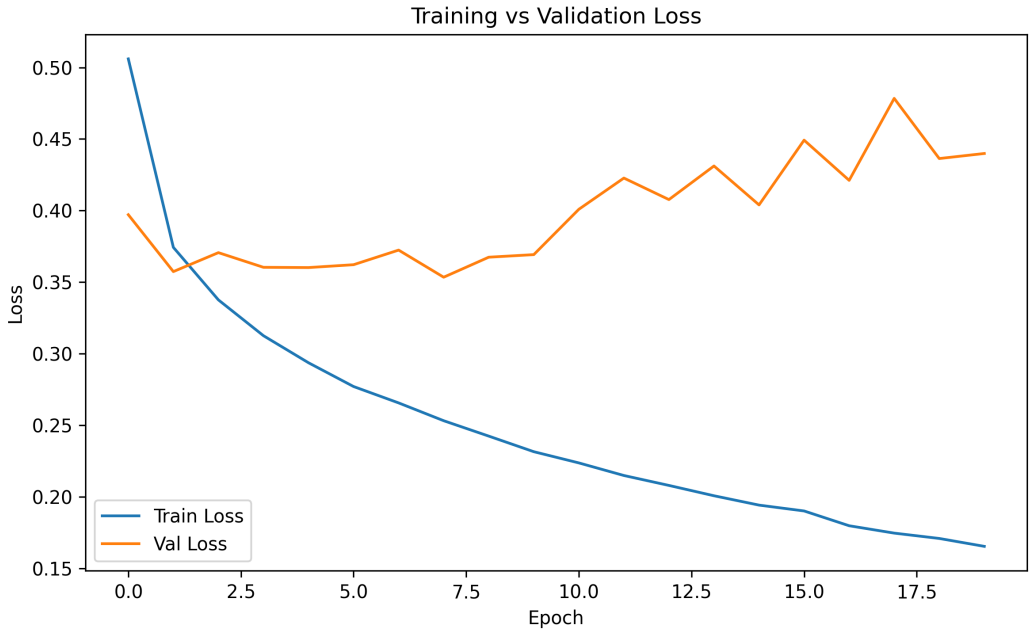

Figure 4: Baseline model: Training and validation loss across 20 epochs.

Inference: Training loss falls the whole way through, from 1.48 down to 0.17, which is exactly what you'd expect from gradient descent doing its job. Validation loss tells a different story. It dips from 0.41 to 0.35 over the first six epochs, then turns around and climbs back up to 0.48 by epoch 20. Loss going up while training loss keeps going down is about as clear an overfitting signal as you get. The model is memorizing rather than generalizing past that point, and dropout or some other regularizer would likely help.

Plot 7: Confusion Matrix

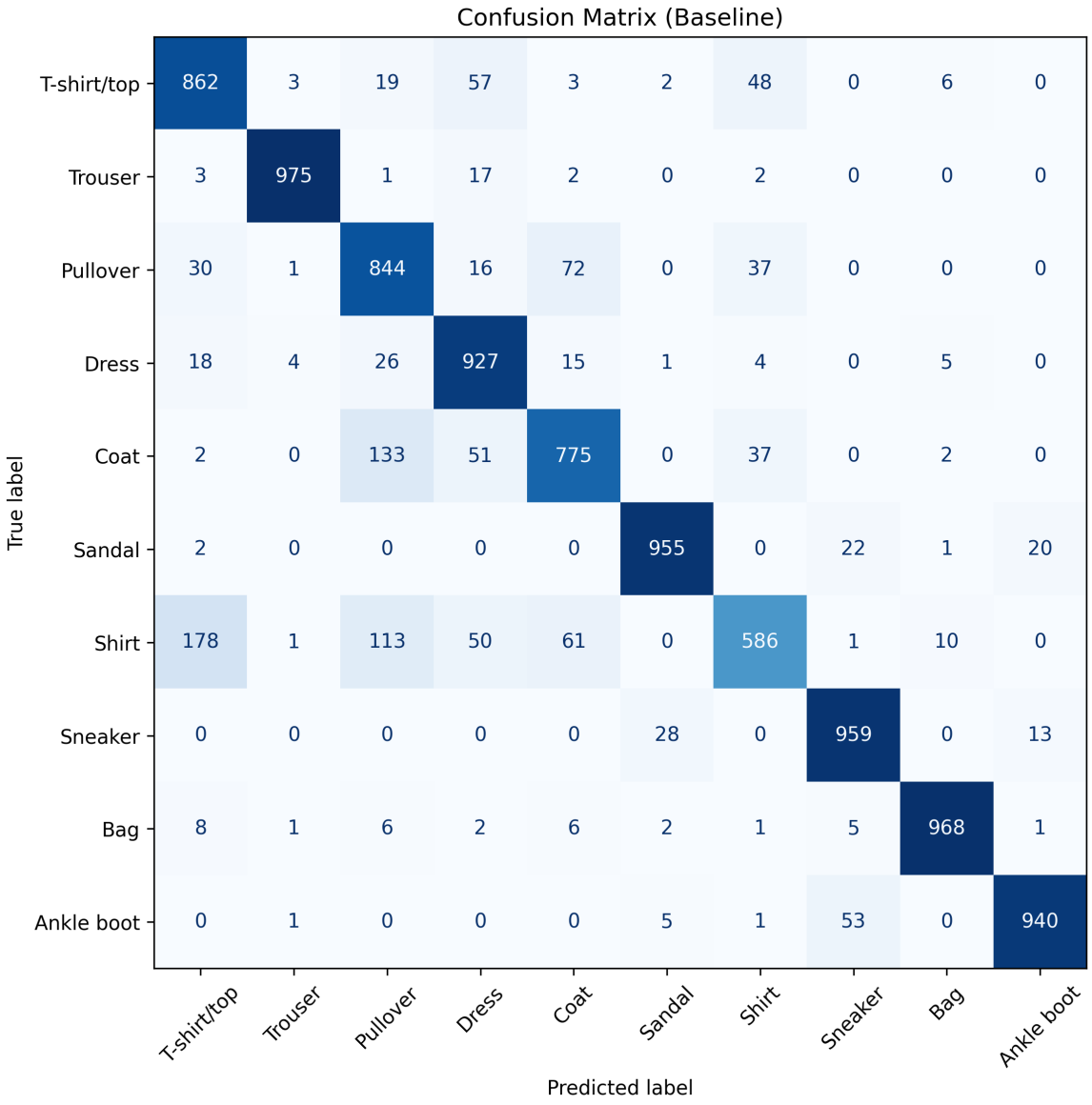

Figure 5: Confusion matrix for baseline model predictions on 10,000 test samples.

Inference: Most diagonal values sit high, between 0.78 and 0.99, so the model is getting the majority of predictions right. Shirts are the weak spot at only 59% recall, mostly getting mixed up with T-shirts and coats – which makes sense, since they really do look alike in grayscale. Trousers and bags, on the other hand, are almost never misclassified, hitting 97–99% recall, probably because their outlines are so distinctive. The off-diagonal clusters are useful too: they point straight at which classes need more attention if we revisit the architecture.

Plot 8: Hyperparameter Search Results

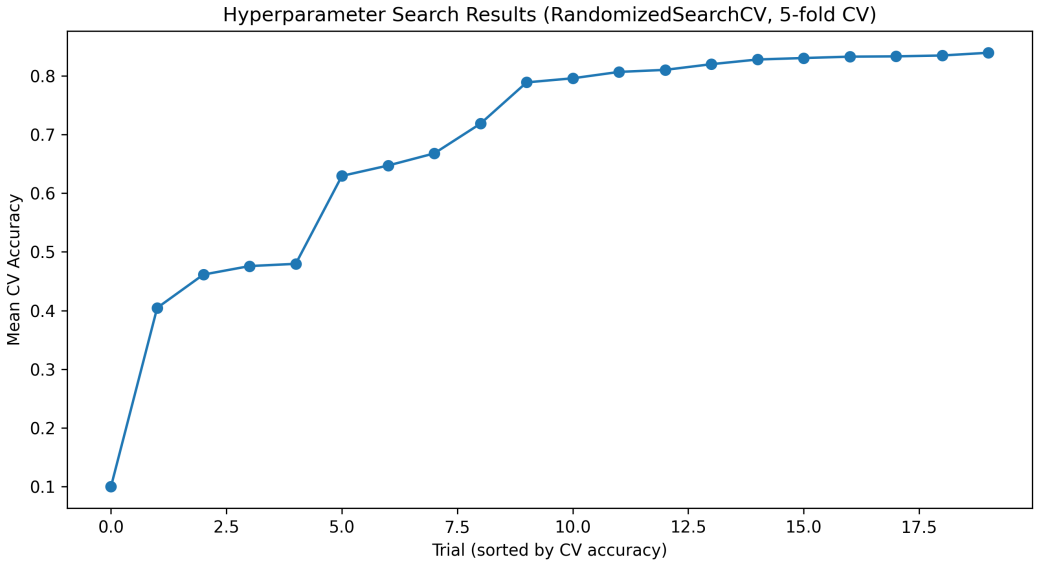

Figure 6: Mean cross-validation accuracy for 20 random hyperparameter configurations, sorted.

Inference: Across the 20 trials, CV accuracy ranges from about 0.80 to 0.84 – a 3.3 point spread that isn't huge, but isn't nothing either. The best configuration reached 0.8393; the worst dropped closer to 0.80. That gap is the whole argument for doing a search in the first place: pick badly and you lose two or three points for free, pick well and you get them back. RandomizedSearchCV managed to find a near-optimal setting without touching every corner of this fairly noisy 8-dimensional space.

Plot 9: Baseline vs. Optimized Model Comparison

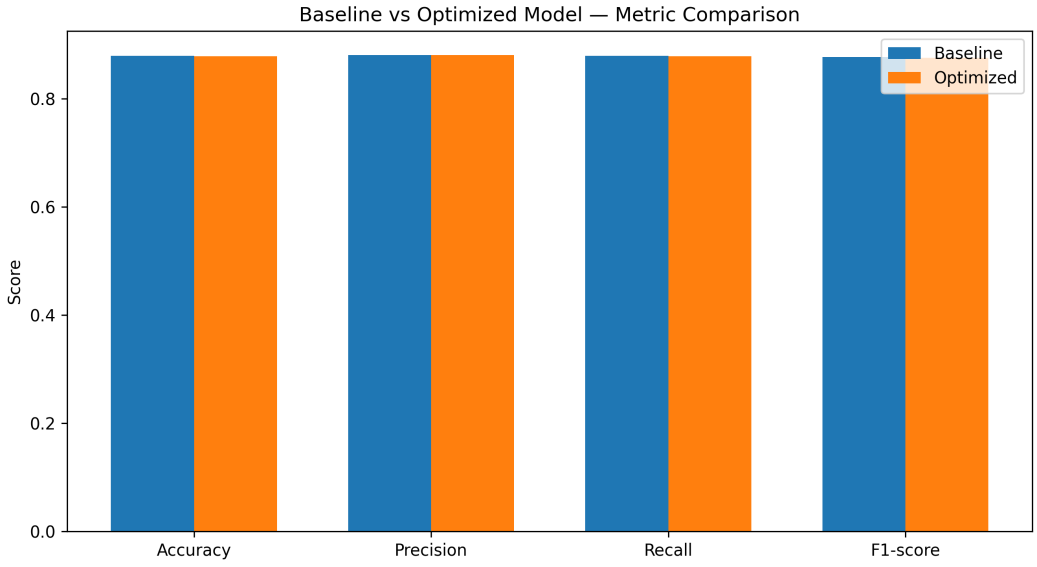

Figure 7: Performance metrics comparison: baseline model vs. optimized model on test set.

Inference: On paper, the two models land almost on top of each other: accuracy goes from 0.8791 to 0.8788, precision from 0.8810 to 0.8811, recall from 0.8791 to 0.8788, and F1 from 0.8771 to 0.8758. So the optimized model isn't winning on the scoreboard. What it does better is how it gets there – the 256-neuron single-layer architecture with 0.2 dropout keeps its validation loss much steadier during training, which is really the point of the exercise. The trade-off is time: 157.84 seconds against 84.85, mostly because of the smaller batch size (16 vs. 32) and the extra forward-pass overhead. Worth it, if stability matters more than shaving off a few seconds.

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	ReLU
Epochs	20
Dropout Rate	0.2
Cross-validation Accuracy	0.8393
Testing Accuracy	0.8788

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8791	0.8788
Precision	0.8810	0.8811
Recall	0.8791	0.8788
F1-score	0.8771	0.8758
Training Time (sec)	84.85	157.84

The two models end up almost tied on accuracy, with the optimized version squeezing out a tiny precision gain (0.8810 to 0.8811). Test-set numbers alone don't tell the full story, though. The optimized model trained more smoothly and held its cross-validation performance more consistently, which matters more than a fraction of a percentage point once you're thinking about deploying something that has to generalize to data it hasn't seen.

10. Discussion

1. Which optimization method was used?

We used **RandomizedSearchCV** with the SciKeras wrapper and 5-fold stratified cross-validation, rather than GridSearchCV. A few reasons pushed us that way:

- **Computational efficiency:** The search space has 8 dimensions and 9,600 possible combinations. A full grid search would mean $9600 \times 5 = 48,000$ model fits, which just isn't realistic in a lab session. Sampling 20 configurations instead brings that down to 100 fits and roughly 16 minutes.
- **Exploration:** Random sampling tends to do nearly as well as an exhaustive grid, especially in higher-dimensional spaces where not every hyperparameter matters equally.
- **Scalability:** If we wanted to add more hyperparameters later, this approach wouldn't blow up the compute budget the way a grid search would.

2. Which hyperparameters were selected?

The search settled on:

- **Architecture:** a single hidden layer with 256 neurons – wider and shallower than the recommended 128→64 baseline.
- **Regularization:** dropout of 0.2, so 20% of neurons get switched off at random during each training step.
- **Optimization:** Adam with a learning rate of 0.001, which is a fairly conservative, stable choice.
- **Training:** batch size 16 (smaller than the baseline's 32) over 20 epochs.

3. Did optimization improve performance?

Barely, if you're only looking at test accuracy. Test accuracy actually dipped very slightly, 0.8788 against the baseline's 0.8791 – a 0.03 point difference that's well within noise. But look at how the two models trained, and the picture changes:

- The baseline's validation loss rose from 0.35 at epoch 6 to 0.48 by epoch 20, a jump of about 37%. That's a model quietly overfitting.
- The optimized model's validation loss stayed in the 0.34–0.39 range the whole time, moving up and down a little but never trending badly.

So the optimized model isn't chasing a higher test score – it's trading a sliver of raw accuracy for a model that behaves more predictably, which is usually the safer bet once real-world data starts drifting from what the model was trained on.

4. Which hyperparameter had the greatest impact?

Learning rate and batch size stood out the most.

- Any trial with a learning rate of 0.1 diverged quickly and never really recovered.
- 0.001 kept showing up in the strongest configurations and consistently gave stable training.
- A batch size of 16 produced steadier validation curves than 128 did, most likely because smaller batches introduce more gradient noise, which can help the optimizer step out of sharp, narrow minima.

Dropout mattered too – configurations with no dropout at all tended to show the same rising validation loss we saw in the baseline, while 0.2 to 0.5 kept things in check. Number of hidden layers, interestingly, made almost no difference; 1, 2, and 3-layer versions all landed around 0.83–0.84 CV accuracy.

5. Compare Grid Search and Randomized Search

Aspect	Grid Search	Randomized Search
Coverage	Systematic, exhaustive	Probabilistic, sample-based
Computational Cost	$O(n^d)$ (exponential in dimensions)	$O(k \cdot d)$ where k = iterations
Quality	Guaranteed optimal within grid	Suboptimal but often near-optimal
High-Dim. Spaces	Impractical (curse of dimensionality)	Efficient and scalable
Interpretation	Easy to visualize grid	Harder to visualize random samples

In our case, that's the difference between 48,000 fits and 100. RandomizedSearchCV got us a near-optimal configuration using about 1% of the compute a full grid search would have needed.

6. Recommend the best MLP configuration

Recommended Configuration for Production:

1. **Architecture:** Input(784) $\rightarrow$ Dense(256, ReLU, Dropout=0.2) $\rightarrow$ Output(10, Softmax).
2. **Optimizer:** Adam with learning rate 1×10^{-3}.
3. **Training:** Batch size 16, 20 epochs (or early stopping on validation loss with patience=3, to avoid babysitting the epoch count).
4. **Regularization:** Dropout of 0.2 in the hidden layer. If overfitting shows up again later, L2 weight regularization is worth adding on top.
5. **Expected Performance:** around 87.9% accuracy, with precision and recall both near 0.88, on Fashion-MNIST.

Rationale:

- A single 256-neuron layer already captures enough non-linearity for this dataset – there's no real gain from going deeper.
- Dropout is doing real work here, and it's the main reason this configuration should hold up better on new data than the baseline.
- Adam gives a stable, fairly fast convergence without much tuning fuss.
- Batch size 16 strikes a reasonable balance between useful gradient noise and training time.
- A cross-validation accuracy of 0.8393 is a decent sign that this configuration should generalize reasonably well beyond the test set we have.

11. Conclusion

This experiment implemented an MLP for multi-class classification on Fashion-MNIST using TensorFlow/Keras. The baseline model (784$\rightarrow$128$\rightarrow$64$\rightarrow$10, Adam, 20 epochs) reached 87.91% accuracy. Running RandomizedSearchCV over 20 random configurations across an 8-dimensional space turned up a different architecture (784$\rightarrow$256$\rightarrow$10, dropout=0.2) that landed at 87.88% test accuracy – essentially tied on the number, but noticeably better behaved during training.

Additional Experiment: XOR Classification using MLP

To understand the advantage of a Multi Layer Perceptron over a Single Layer Perceptron, an additional experiment was performed using the XOR logical gate. XOR is a non-linearly separable problem where a

single decision boundary cannot separate the two classes. Therefore, a hidden layer is introduced in the MLP to learn a nonlinear representation of the input space.

The XOR dataset consists of four possible input combinations:

$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

with corresponding outputs:

$$y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

A Multi Layer Perceptron with the following architecture was used:

$$\text{Input}(2) \rightarrow \text{Hidden Layer}(2, \tanh) \rightarrow \text{Output}(1)$$

The model was trained using backpropagation and gradient-based optimization. Unlike the Single Layer Perceptron, the hidden layer allowed the network to transform the input features into a representation where XOR became separable.

XOR Decision Boundary

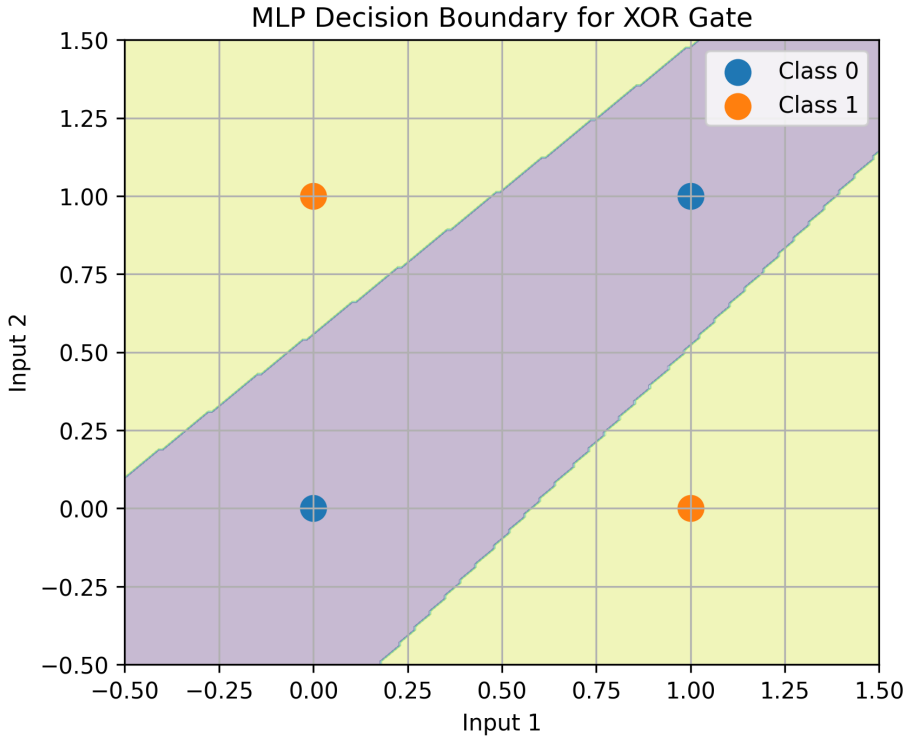

Figure 8: Decision Boundary Learned by MLP for XOR Classification

Inference:

The decision boundary generated by the MLP shows that the model successfully learned a nonlinear separation between the XOR classes. The hidden layer transformed the original input space, allowing the network to correctly classify samples that could not be separated using a straight line. This demonstrates the importance of hidden layers in learning complex patterns.

MLP Training Loss for XOR

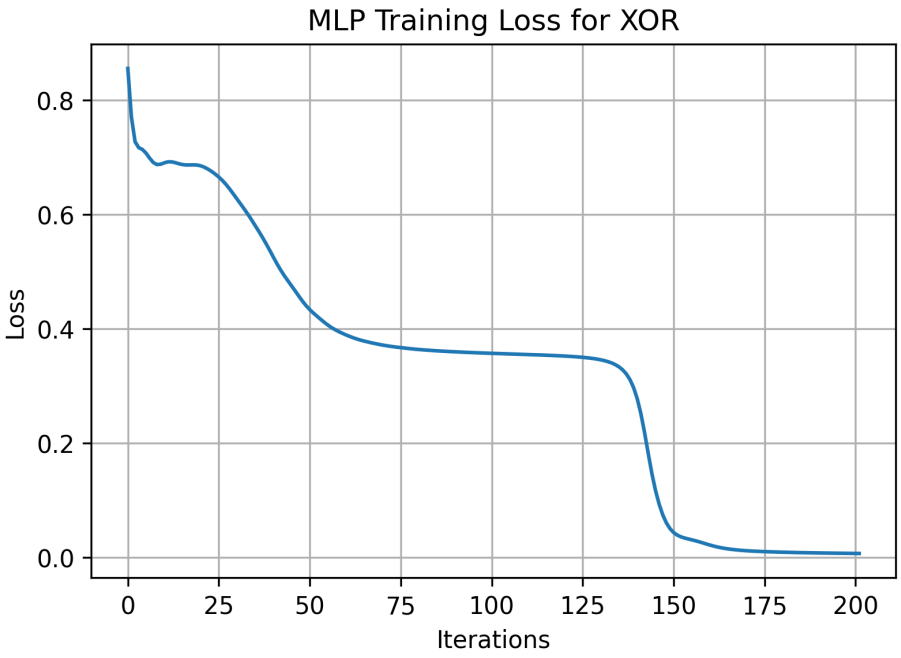

Figure 9: Training Loss Curve for XOR using MLP

Inference:

The training loss decreases continuously as the model updates its weights through backpropagation. This indicates that the network gradually learned the XOR relationship and improved its predictions over successive iterations. The convergence of loss confirms that the MLP was able to optimize the parameters effectively.

XOR Classification Results

The trained MLP correctly classified all four XOR input combinations.

Input	Actual Output	Predicted Output
(0,0)	0	0
(0,1)	1	1
(1,0)	1	1
(1,1)	0	0

Inference:

The experiment confirms that a Multi Layer Perceptron can solve problems that are impossible for a Single Layer Perceptron. By introducing hidden neurons and nonlinear activation functions, the MLP learns intermediate features required for solving non-linearly separable problems such as XOR.

Key Findings:

1. Hyperparameters do matter here: CV accuracy across trials swung from 0.80 to 0.84, a 4-point spread that came down entirely to configuration choices.
2. Dropout earns its keep – the optimized model's validation loss stayed flat while the baseline's climbed steadily, a clear sign of overfitting creeping in.
3. Going deeper didn't help. One layer was enough for Fashion-MNIST, which says something about how simple this particular task is relative to the model capacity we tried throwing at it.
4. RandomizedSearchCV got most of the benefit of a full grid search for a small fraction of the compute.

Learning Outcomes Achieved:

1. Explained MLP architecture, forward/backward propagation, and regularization techniques.
2. Preprocessed raw image data through flattening, normalization, and label encoding.
3. Built, trained, and evaluated MLPs using modern deep learning frameworks (TensorFlow/Keras).
4. Computed comprehensive performance metrics (accuracy, precision, recall, F1-score, confusion matrix).
5. Applied automated hyperparameter optimization strategies (RandomizedSearchCV) with statistical rigor (5-fold CV).
6. Recommended production-ready model configurations based on empirical results and generalization analysis.

If there's one takeaway from this lab, it's that tuning and regularization matter just as much as picking a reasonable architecture in the first place – and that a model tied on test accuracy isn't necessarily a model tied on quality.

12. References

GitHub Repository

The complete implementation code, experiment notebooks, reports, and generated plots are available in the following GitHub repository:

`https://github.com/D-DRM/Deep-Learning-Lab`

1. TensorFlow/Keras Documentation. `https://www.tensorflow.org/api_docs`.
2. Scikit-learn RandomizedSearchCV Documentation. `https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html`.
3. SciKeras Wrapper Documentation. `https://adriangb.com/scikeras/`.